https://github.com/pattosz/TM-95123

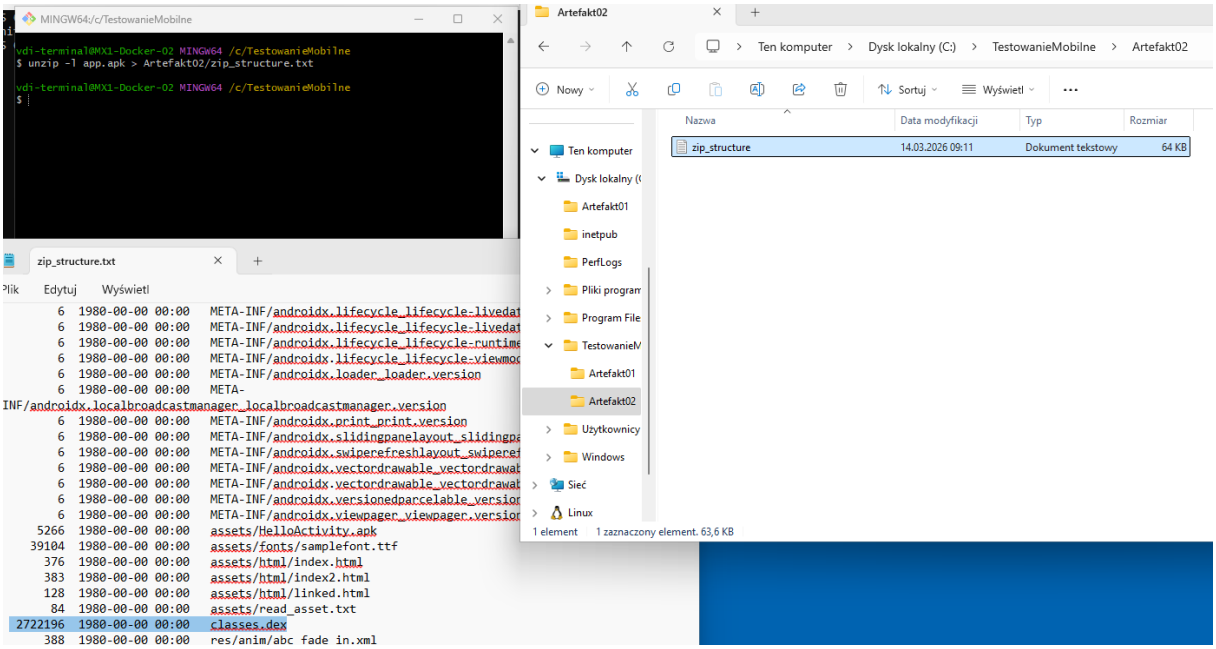
2.1. Struktura archiwum (unzip -l) Na zajęciach używamy: ApiDemos-debug.apk Klasyczna aplikacja testowa Androida.

Do pobrania z: https://github.com/appium/appium/blob/master/packages/appium/sample-code/apps/ApiDemos-debug.apk

Nie musimy jej oznaczać encją – to tylko plik testowy.

Student wrzuca plik do katalogu głównego repo.

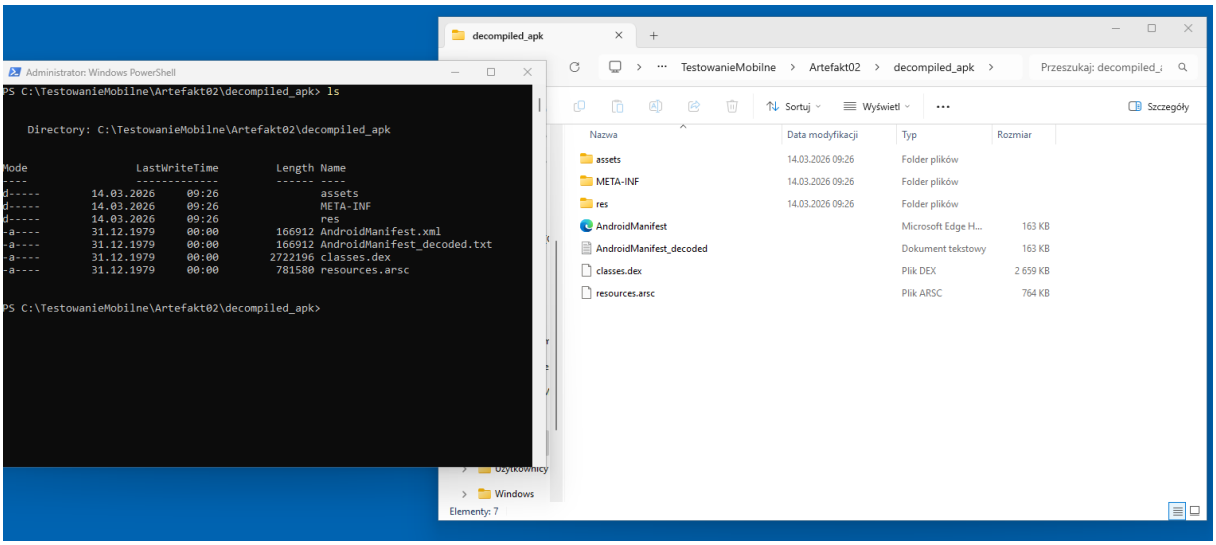
Dla łatwiejszego przetwarzania możemy zmienić nazwę na app.apk



2.2. Dekompilacja APK Artefakt 2.2. Artefakt02/decompiled_apk/

SCREENSHOT: Widok komendy ls (MAC/ LINUX lub dir (Windows) gdzie widać folder decompiled_apk oraz plik AndroidManifest_decoded.txt. 13

Co student robi realnie? ✓ dekompiluje aplikację ✓ analizuje manifest ✓ analizuje strukturę APK ✓ rozumie jak działa aplikacja Android

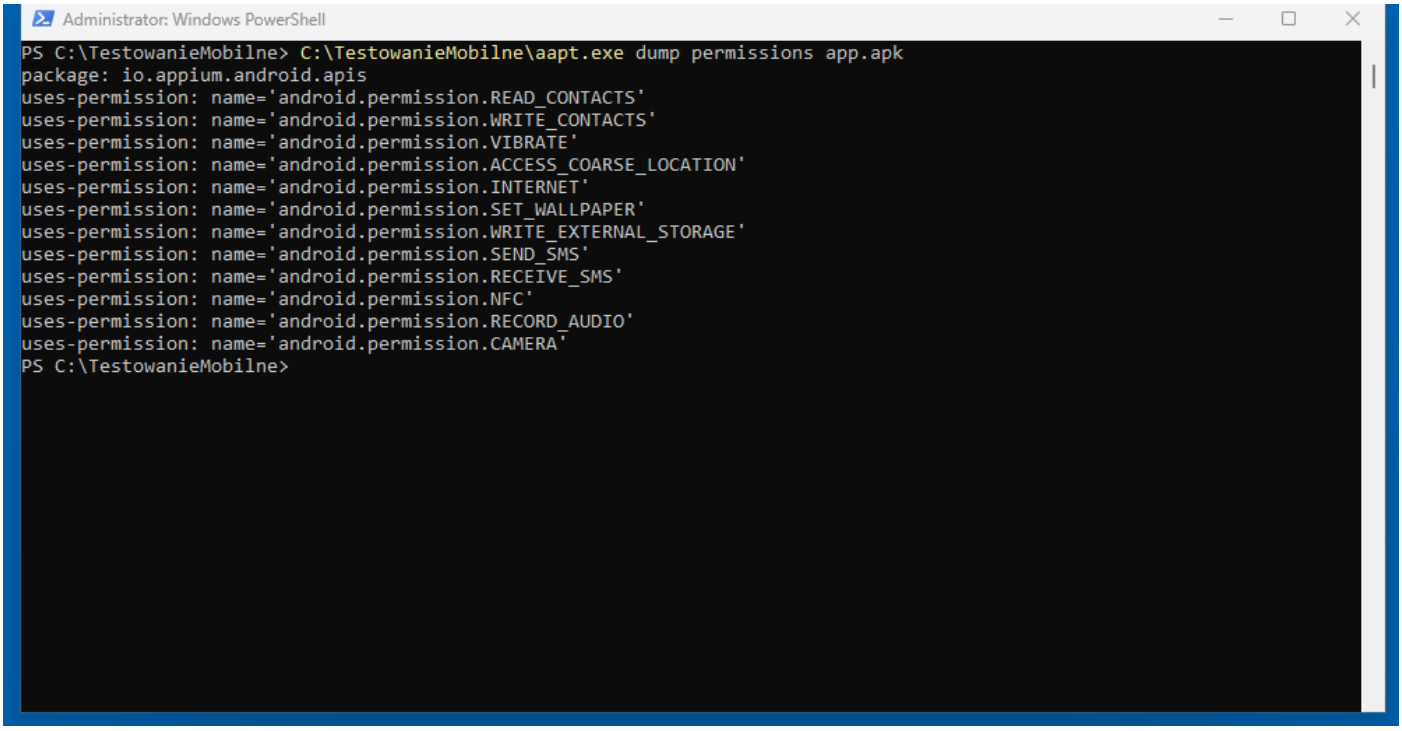


2.3. Uprawnienia (Permissions) Komenda: aapt dump permissions

SCREENSHOT: Lista uprawnień zaczynających się od uses-permission.

Czego się uczę: Analizy tzw. "powierzchni ataku" (Attack Surface) oraz zasady minimalnych uprawnień.

Wartość dla testera: Student uczy się zadawać pytanie: "Po co kalkulatorowi dostęp do kontaktów i mikrofonu?". Nadmiarowe uprawnienia to krytyczny błąd bezpieczeństwa. Tester musi umieć ocenić, czy uprawnienia takie jak RECORD_AUDIO czy ACCESS_FINE_LOCATION są uzasadnione biznesowo, czy są luką, którą może wykorzystać malware.



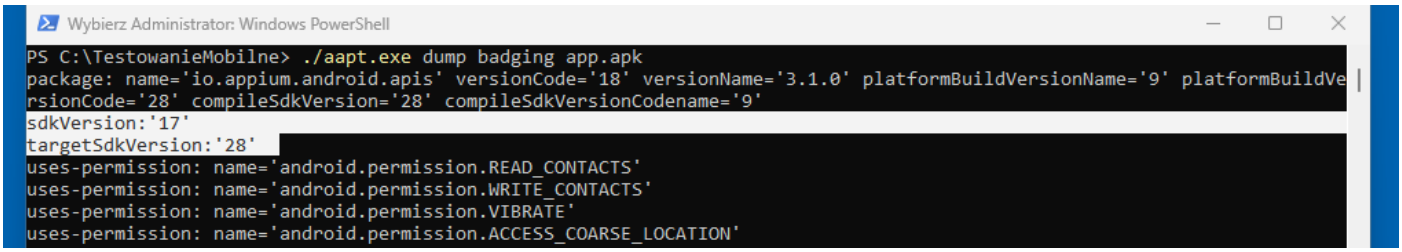
2.4. Poziom API (api_level)

Komenda: aapt dump badging

SCREENSHOT: Wynik pokazujący SdkVersion i targetSdkVersion

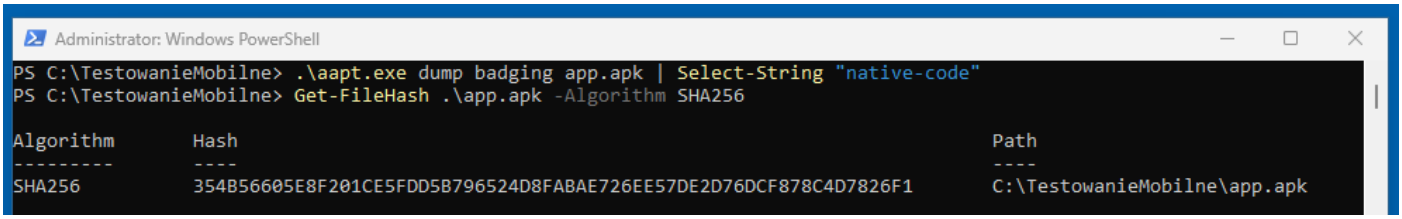
Czego się uczę: Zarządzania kompatybilnością i bezpieczeństwem systemu operacyjnego.

Wartość dla testera: To klucz do zrozumienia ryzyka. Jeśli minSdkVersion jest bardzo niskie (np. 17), aplikacja będzie działać na starych Androidach, które mają tysiące znanych luk np. Stagefright). Z kolei niskie targetSdkVersion pozwala aplikacji omijać nowoczesne mechanizmy ochrony Androida (np. prośby o uprawnienia w czasie rzeczywistym). Tester wie wtedy, że aplikacja jest "dziurawa" z definicji.



2.5. Architektura (ABI) i Hash SHA256 Komenda: aapt dump badging app.apk | grep native-code"; shasum -a 256 app.apk

SCREENSHOT: Wynik native-code oraz hash wygenerowany przez Twój komputer.

Czego się uczę: Zapewnienia integralności (Integrity) i identyfikacji technologii.

Wartość dla testera: Hash: Student uczy się, że w bezpieczeństwie nie ufa się nazwie pliku (app.apk), tylko jego "odciskowi palca". Jeśli hash się nie zgadza z oryginałem, tester wie, że ktoś mógł wstrzyknąć do aplikacji złośliwy kod (trojanizowanie APK). Native Code: Sprawdzenie architektury mówi testerowi, czy aplikacja używa bibliotek pisanych w C/C++ (NDK). Jeśli tak, to wiadomo, że trzeba szukać błędów związanych z zarządzaniem pamięcią (np. Buffer Overflow), a nie tylko błędów w Javie.